

Meridian Service Desk — Triage & SOP Agent

An agentic AI system that drafts service-desk replies grounded in documented procedure — and cannot send anything without a human.
Aman Mishra · IIM Udaipur · Building with Agentic AI, capstone · September 2026

Live: <paste your Render URL here>

Code: <https://github.com/AManMishra20/TicketAgent>

What this is

An internal IT service desk takes roughly 1,200 tickets a month. Most are repeats of something already solved and documented — but documentation only helps if someone remembers it exists. This closes that loop both ways.

Agent A (resolver) reads a new ticket, looks up the requester's support tier, searches the procedure library, and drafts a reply that cites the procedure by its document ID. **Agent B (author)** takes the opposite case: when a ticket was resolved that no procedure covered, it reads how the human actually fixed it and drafts the missing procedure.

The problem is modelled on enterprise IT support work — Jira for tickets, Confluence for procedures. In this build those are stand-ins: a local ticket store and a folder of markdown, so the demonstration has no external dependency. The interface labels them as stand-ins rather than claiming a live connection.

The core design decision

The human-approval gate is a **code boundary, not a prompt instruction**. Asking a model to seek permission is a request it can be argued out of — including by text inside a ticket. Here it is structural, in four independent layers:

1. Rule	Stated in CLAUDE.md and the system prompt. A request.
2. Gate	Runs in code after the model speaks, reading the ticket and requester record — not the model's opinion.
3. Boundary	The model is never given a write-capable tool. Its only terminal move produces a proposal object.
4. Hook	Refuses any code edit that would widen the tool surface.

Four conditions always stop a draft for a person: a request for access or permissions, critical severity, a Gold-tier requester (an unknown requester counts as Gold), and — the important one — **no procedure covering the ticket**. The agent reporting that it cannot answer is a correct outcome, not a failure: it routes the ticket to Agent B to write what is missing.

This guards against the failure that made Air Canada liable for its chatbot — a confident, well-formed policy that did not exist. Citations are verified in code after the model speaks; an ID that does not exist is stripped and the proposal downgraded.

How it works

Read	The agent picks its own inputs: the ticket, the requester record, the procedure library. Nothing is pasted in for it.
Score	A match score computed in code, not the model's opinion of its own confidence. The reviewer sees the same number.
Propose	The model's only terminal move: a structured object, not an action. A second instance writes the reply from three inputs — ticket, tier, clause.
Hold	The gate re-decides independently and stamps HOLD where any condition applies.
Approve	A person decides. This is the only path that writes anything.

Checking it yourself

The repository runs offline with no API key. Every figure opposite is reproducible:

```
pip install -r requirements.txt
python -m src.ticket_agent.seed --seed 42
pytest -q
uvicorn src.ticket_agent.web.app:app
```

The corpus is seeded deterministically: 53 tickets, 6 procedures, and five defects planted on purpose, because real queues carry all five.

Using the website

- 1. Overview.** Three panels show the systems: Jira (where tickets come from), Confluence (where procedures live), and the agent itself. Press **Activate** — the agent panel turns green and begins picking up tickets on its own.
- 2. Review queue.** Each row is something the agent wants to do and has not done. A red spine means it stopped; green means it is ready to send.
- 3. Open a red row.** It carries a HOLD: HUMAN REVIEW stamp and states which condition stopped it. Try approving it with the note field empty — **it refuses to send**. That is the system working.
- 4. Open a green row.** The right column shows which procedure matched and the score it matched at. Edit the reply if needed, then approve — only now does it appear on the ticket.
- 5. Back to Overview.** The decision appears in the counts. Edits are kept alongside what the agent originally wrote: repeated edits of the same kind indicate a missing rule, not three separate mistakes.

Browse tickets and procedures from the left rail at any time. **Deactivate** stops the agent.

Measured results

Retrieval precision@1 (38 covered tickets)	100%
Uncovered tickets wrongly matched	0
Open tickets held for a human / ready	29 / 10
Correct matches routed to a human anyway	5 of 38
Automated tests, offline, no API key	83 passing

The match threshold (0.37) was calibrated against known ground truth, not guessed. It costs 13% recall and buys zero false citations — a deliberate asymmetry, since a false refusal costs a reviewer thirty seconds while a false citation is a policy stated in writing to someone who will act on it.

Declaration of AI use

This project was built with Claude (Anthropic), used as a coding agent throughout. The extent of that use is material and is stated in full.

Generated by AI: substantially all source code, the test suite, the interface and its styling, the repository documentation, and the text of this document. AI also proposed the retrieval-scoring method, calibrated the confidence threshold, and identified the thread-safety defect in the background worker.

My contribution: defining the problem from my own service-desk experience at TCS; the governing constraint that the approval gate must be structural rather than a prompt instruction; the design decisions the agent raised for a human to settle — data backend, deployment target, interface scope, how the systems are labelled; direction of the build and review of each stage; and deployment.

Verification: every factual claim above is reproducible from the repository. The tests run offline without an API key.

Stated limitation: the results above were produced in demo mode, where the language model is replaced by a fixed decision procedure over the same retrieval results. The safety architecture, retrieval and interface figures are real; live-model accuracy and token cost are **not measured**, and are recorded as unmeasured in docs/EVIDENCE.md rather than estimated.